

Mosquito

The *Culex computarus* mosquito exists in a two dimensional square world. It is nocturnal. At night it flies at constant speed (1 meter per second) in a straight line for a short segment (1 meter), and then randomly chooses a new direction for the next segment. Each mosquito moves independently of other mosquitos, and they are small enough that they do not collide when their paths cross. The world also includes obstacles, which are straight walls of zero width. When a mosquito encounters an obstacle or world boundary, it chooses a new random direction away from the obstacle/boundary and starts a new 1 meter segment.

Your job is to corral the mosquitos for collection by PPS Mosquito Research Inc. They have a circular device with a diameter of 1 meter that will capture any mosquitos that fly over it. Your tool for this job is a collection of lights that you can place around the world and turn on and off over time. Mosquitos are attracted to the lights, and their flying behavior changes.

A mosquito will be attracted to the brightest light. Because all lights have the same intensity, this means they are attracted to the nearest light that is switched on and visible in a straight line from the mosquito. (Obstacles obscure lights if they are between a mosquito and a light.) However, a mosquito is not attracted at all to a dim light -- lights that are more than 20 meters away have no effect on mosquitos; after all, flying towards a full moon would probably not be a good evolutionary strategy.

When a mosquito is about to choose a new direction, it will choose a direction biased by the nearest light (within 20 meters). The new direction will be within 30 degrees of the direction of the light. For example, suppose the mosquito is at (50,50), that the light is at (50,60), and that there are no obstacles. Then the new direction will be a random angle chosen uniformly from between 60 degrees and 120 degrees using conventional polar coordinates. The mosquito will travel 1 meter in the chosen direction, and then repeat the process of selecting a direction. When a mosquito gets very close to a light, this process may take the mosquito past the light. (We'll assume the light is a point source, and that mosquitoes don't collide with the light.)

A light may gather a swarm of mosquitoes. The only way to "release" those mosquitoes is to turn off the light. You can program a light to turn on and off at regular intervals. Each light has three floating point parameters measured in seconds: d , t , and s . d specifies the interval between lighting cycles. t specifies how long a light stays on within a cycle (t is no more than d). s specifies a start time at which the light begins to operate. For example, if $d=10$, $t=6$ and $s=7$, then the light turns on at times 7, 17, 27, etc., and off at 13, 23, 33, etc. By choosing the parameters appropriately, you should be able to coordinate the different lights to achieve global behaviors.

You will be given the following information describing the world. The world is assumed to be a cartesian square with opposing corners at (0,0) and (100,100).

- The endpoints of zero or more obstacles. You may assume that the obstacles do not partition the world.
- A number L of lights you get to use.

Based on this information, your program should specify the location of the collector, and the location and parameters for each light. The collector cannot overlap an obstacle, light, or a world boundary. The score of your layout is the time taken to collect 50% of the mosquitoes. (The actual number of mosquitoes will be large, at least one thousand; we'll try to put as many as possible in the simulation subject to being able to simulate the system in reasonable time.) Mosquitoes are initially distributed randomly within the world. We may do several runs and average them to compensate for variability due to the randomness.

Your program for placing the lights and collector does not need to be interactive. In fact, it is acceptable for you to try out different options by simulating them yourselves. We'll put some time limit on your program, such as one hour of elapsed time on a single CPU. (We'll discuss what this limit should be in class.)

In addition to solving the worlds, each group will be asked to build worlds with interesting obstacle configurations.